

ATTRIBUTE PROTOCOL (ATT)

This specification defines the Attribute Protocol; a protocol for discovering, reading, and writing attributes on a peer device



CONTENTS

1	Introduction	2172
1.1	Scope	2172
1.2	Conformance	2172
1.3	Conventions.....	2172
2	Protocol Overview	2173
3	Protocol Requirements	2174
3.1	Introduction	2174
3.2	Basic Concepts.....	2174
3.2.1	Attribute Type.....	2174
3.2.2	Attribute Handle	2174
3.2.3	Attribute Handle Grouping	2175
3.2.4	Attribute Value.....	2175
3.2.5	Attribute Permissions	2175
3.2.6	Control-Point Attributes	2177
3.2.7	Protocol Methods	2177
3.2.8	Exchanging MTU Size	2177
3.2.9	Long Attribute Values	2178
3.2.10	Atomic Operations	2178
3.3	Attribute PDU	2178
3.3.1	Attribute PDU Format.....	2179
3.3.2	Sequential Protocol.....	2180
3.3.3	Transaction	2181
3.4	Attribute Protocol PDUs.....	2182
3.4.1	Error Handling.....	2182
3.4.1.1	Error Response	2182
3.4.2	MTU Exchange	2184
3.4.2.1	Exchange MTU Request	2184
3.4.2.2	Exchange MTU Response	2184
3.4.3	Find Information	2186
3.4.3.1	Find Information Request	2186
3.4.3.2	Find Information Response	2187
3.4.3.3	Find By Type Value Request	2189
3.4.3.4	Find By Type Value Response	2190
3.4.4	Reading Attributes.....	2191
3.4.4.1	Read By Type Request.....	2191
3.4.4.2	Read By Type Response.....	2193
3.4.4.3	Read Request	2194
3.4.4.4	Read Response.....	2194

Attribute Protocol (ATT)

3.4.4.5	Read Blob Request	2195
3.4.4.6	Read Blob Response	2196
3.4.4.7	Read Multiple Request	2197
3.4.4.8	Read Multiple Response	2198
3.4.4.9	Read by Group Type Request	2198
3.4.4.10	Read by Group Type Response	2200
3.4.5	Writing Attributes	2201
3.4.5.1	Write Request	2201
3.4.5.2	Write Response	2202
3.4.5.3	Write Command	2203
3.4.5.4	Signed Write Command	2204
3.4.6	Queued Writes	2205
3.4.6.1	Prepare Write Request	2205
3.4.6.2	Prepare Write Response	2207
3.4.6.3	Execute Write Request	2207
3.4.6.4	Execute Write Response	2208
3.4.7	Server Initiated	2209
3.4.7.1	Handle Value Notification	2209
3.4.7.2	Handle Value Indication	2209
3.4.7.3	Handle Value Confirmation	2210
3.4.8	Attribute Opcode Summary	2211
3.4.9	Attribute PDU Response Summary	2213
4	Security Considerations	2216



1 INTRODUCTION

1.1 SCOPE

The attribute protocol allows a device referred to as the server to expose a set of attributes and their associated values to a peer device referred to as the client. These attributes exposed by the server can be discovered, read, and written by a client, and can be indicated and notified by the server.

~~1.2 CONFORMANCE~~

This Section was [amended](#) on 2024-06-11 by the Bluetooth Core Specification Amendment: Core Configuration and Compliance Updates. See [page 0](#) for more information.

~~If conformance to this protocol is claimed, all capabilities indicated as mandatory for this protocol shall be supported in the specified manner (process mandatory). This also applies to all optional and conditional capabilities for which support is indicated. All mandatory capabilities, and optional and conditional capabilities for which support is indicated, are subject to verification as part of the Bluetooth qualification program.~~

1.3 CONVENTIONS

In this specification PDU names appear in italics. Error codes defined in [Table 3.3](#) (see [Section 3.4.1.1](#)) appear in « » (e.g. «Attribute Not Found»). Other names such as parameters appear in Roman text.



2 PROTOCOL OVERVIEW

The attribute protocol defines two roles; a server role and a client role. It allows a server to expose a set of attributes to a client that are accessible using the attribute protocol.

An attribute is a discrete value that has the following three properties associated with it:

- a) attribute type, defined by a UUID
- b) attribute handle
- c) a set of permissions that are defined by each higher layer specification that utilizes the attribute; these permissions cannot be accessed using the Attribute Protocol.

The attribute type specifies what the attribute represents. Bluetooth SIG defined attribute types are defined in the Bluetooth SIG [Assigned Numbers](#) page, and used by an associated higher layer specification. Non-Bluetooth SIG attribute types may also be defined.

The attribute handle uniquely identifies an attribute on a server, allowing a client to reference the attribute in read or write requests; see [Section 3.4.4](#), [Section 3.4.5](#), and [Section 3.4.6](#). It allows a client to uniquely identify the attribute being notified or indicated, see [Section 3.4.7](#). Clients are able to discover the handles of the server's attributes; see [Section 3.4.3](#). Permissions may be applied to an attribute to prevent applications from obtaining or altering an attribute's value. An attribute may be defined by a higher layer specification to be readable or writable or both, and may have additional security requirements. For more information, see [Section 3.2.5](#).

A client may send attribute protocol requests to a server, and the server shall respond to all requests that it receives. A device can implement both client and server roles, and both roles can function concurrently in the same device and between the same devices. There shall be only one instance of a server on each Bluetooth device; this implies that the attribute handles shall be identical for all supported bearers. For a given client, the server shall have one set of attributes. The server can support multiple clients. Note: Multiple services may be exposed on a single server by allocating separate ranges of handles for each service. The discovery of these handle ranges is defined by a higher layer specification.

The attribute protocol has notification and indication capabilities that provide an efficient way of sending attribute values to a client without the need for them to be read; see [Section 3.3](#).



3 PROTOCOL REQUIREMENTS

3.1 INTRODUCTION

Each attribute has an attribute type that identifies, by means of a UUID (Universally Unique IDentifier), what the attribute represents so that a client can understand the attributes exposed by a server. Each attribute has an attribute handle that is used for accessing the attribute on a server, as well as an attribute value.

An attribute value is accessed using its attribute handle. The attribute handles are discovered by a client using attribute protocol PDUs (Protocol Data Unit). Attributes that have the same attribute type may exist more than once in a server. Attributes also have a set of permissions that controls whether they can be read or written, or whether the attribute value shall be sent over an encrypted link. Security aspects of the attribute protocol are defined in [Section 4](#).

3.2 BASIC CONCEPTS

3.2.1 Attribute Type

A universally unique identifier (UUID) is used to identify every attribute type. A UUID is considered unique over all space and time. A UUID can be independently created by anybody and distributed or published as required. There is no central registry for UUIDs, as they are based off a unique identifier that is not duplicated. The attribute protocol allows devices to identify attribute types using UUIDs regardless of the local handle used to identify them in a read or write request.

Universal unique identifiers are defined in SDP [\[Vol 3\] Part B, Section 2.5.1](#).

All 32-bit Attribute UUIDs shall be converted to 128-bit UUIDs when the Attribute UUID is contained in an ATT PDU.

3.2.2 Attribute Handle

An attribute handle is a 16-bit value that is assigned by each server to its own attributes to allow a client to reference those attributes. An attribute handle shall not be reused while an ATT Bearer exists between a client and its server.

Attribute handles on any given server shall have unique, non-zero values. Attributes are ordered by attribute handle.

An attribute handle of value 0x0000 is reserved for future use. An attribute handle of value 0xFFFF is known as the maximum attribute handle.



Note: Attributes can be added or removed while an ATT Bearer is active, however, an attribute that has been removed cannot be replaced by another attribute with the same handle while an ATT Bearer is active.

3.2.3 Attribute Handle Grouping

Grouping is defined by a specific attribute placed at the beginning of a range of other attributes that are grouped with that attribute, as defined by a higher layer specification. Clients can request the first and last handles associated with a group of attributes.

3.2.4 Attribute Value

An attribute value is an octet array that may be either fixed or variable length. For example, it can be a one octet value, or a four octet integer, or a variable length string. An attribute may contain a value that is too large to transmit in a single PDU and can be sent using multiple PDUs. The values that are transmitted are opaque to the attribute protocol. The encoding of these octet arrays is defined by the attribute type.

When transmitting attribute values in a request, a response, a notification or an indication, the attribute value length is not sent in any field of the PDU. The length of a variable length field in the PDU is implicitly given by the length of the packet that carries this PDU. This implies that:

- a) only one attribute value can be placed in a single request, response, notification or indication unless the attribute values have lengths known by both the server and client, as defined by the attribute type.
- b) This attribute value will always be the only variable length field of a request, response, notification or indication.
- c) The bearer protocol (e.g. L2CAP) preserves datagram boundaries.

Note: Some responses include multiple attribute values, for example when client requests multiple attribute reads. For the client to determine the attribute value boundaries, the attribute values must have a fixed size defined by the attribute type.

3.2.5 Attribute Permissions

An attribute has a set of permission values associated with it. The permissions associated with an attribute specifies that it may be read and/or written. The permissions associated with the attribute specifies the security level required for read and/or write access, as well as notification and/or indication. The permissions of a given attribute are defined by a higher layer specification, and are not discoverable using the attribute protocol.



If access to a secure attribute requires an authenticated link, and the client is not already authenticated with the server with sufficient security, then an error response shall be sent with the error code «Insufficient Authentication». When a client receives this error code it may try to authenticate the link, and if the authentication is successful, it can then access the secure attribute.

If access to a secure attribute requires an encrypted link, and the link is not encrypted, then an error response shall be sent with the error code «Insufficient Encryption». When a client receives this error code it may try to encrypt the link and if the encryption is successful, it can then access the secure attribute.

If access to a secure attribute requires an encrypted link, and the link is encrypted but with an encryption key size that is too short for the level of security required, then an error response shall be sent with the error code «Insufficient Encryption Key Size». When a client receives this error code it may try to encrypt the link with a larger key size, and if the encryption is successful, it can then access the secure attribute.

Attribute permissions are a combination of access permissions, encryption permissions, authentication permissions and authorization permissions.

The following access permissions are possible:

- Readable
- Writeable
- Readable and writable

The following encryption permissions are possible:

- Encryption required
- No encryption required

The following authentication permissions are possible:

- Authentication Required
- No Authentication Required

The following authorization permissions are possible:

- Authorization Required
- No Authorization Required

Encryption, authentication, and authorization permissions can have different possibilities; for example, a specific attribute could require a particular kind of authentication or a certain minimum encryption key length. An attribute can have several combinations of permissions that apply; for example, a specific attribute could allow any of the following:

- Read if encrypted (authentication not required)



- Write if authenticated and encrypted
- Read or write if authenticated and authorized (irrespective of encryption)

Access permissions are used by a server to determine if a client can read and/or write an attribute value.

Authentication permissions are used by a server to determine if an authenticated physical link is required when a client attempts to access an attribute. Authentication permissions are also used by a server to determine if an authenticated physical link is required before sending a notification or indication to a client.

Authorization permissions determine if a client needs to be authorized before accessing an attribute value.

3.2.6 Control-Point Attributes

Attributes that cannot be read, but can only be written, notified or indicated are called control-point attributes. These control-point attributes can be used by higher layers to enable device specific procedures, for example the writing of a command or the indication when a given procedure on a device has completed.

3.2.7 Protocol Methods

The attribute protocol uses methods defined in [Section 3.4](#) to find, read, write, notify, and indicate attributes. A method is categorized as either a command, a request, a response, a notification, an indication, or a confirmation method; see [Section 3.3](#). Some attribute protocol PDUs can also include an Authentication Signature, to allow authentication of the originator of this PDU without requiring encryption. The method and signed bit are known as the opcode.

3.2.8 Exchanging MTU Size

ATT_MTU is defined as the maximum size of any packet sent between a client and a server. A higher layer specification defines the default ATT_MTU value.

The client and server may optionally exchange the maximum size of a packet that can be received using the Exchange MTU Request and Response PDUs. Both devices then use the minimum of these exchanged values for all further communication (see [Section 3.4.2](#)).

A device that is acting as a server and client at the same time shall use the same value for Client Rx MTU and Server Rx MTU.

The ATT_MTU value is a per ATT Bearer value. Note: A device with multiple ATT Bearers may have a different ATT_MTU value for each ATT Bearer.



3.2.9 Long Attribute Values

The longest attribute that can be sent in a single packet is (ATT_MTU-1) octets in size. At a minimum, the Attribute Opcode is included in an Attribute PDU.

An attribute value may be defined to be larger than (ATT_MTU-1) octets in size. These attributes are called long attributes.

To read the entire value of an attributes larger than (ATT_MTU-1) octets, the read blob request is used. It is possible to read the first (ATT_MTU-1) octets of a long attribute value using the read request.

To write the entire value of an attribute larger than (ATT_MTU-3) octets, the prepare write request and execute write request is used. It is possible to write the first (ATT_MTU-3) octets of a long attribute value using the write request.

It is not possible to determine if an attribute value is longer than (ATT_MTU-3) octets using this protocol. A higher layer specification will state that a given attribute can have a maximum length larger than (ATT_MTU-3) octets.

The maximum length of an attribute value shall be 512 octets.

Note: The protection of an attribute value changing when reading the value using multiple attribute protocol PDUs is the responsibility of the higher layer.

3.2.10 Atomic Operations

The server shall treat each request or command as an atomic operation that cannot be affected by another client sending a request or command at the same time. If a physical link is disconnected for any reason (user action or loss of the radio link), the value of any modified attribute is the responsibility of the higher layer specification.

Long attributes cannot be read or written in a single atomic operation.

3.3 ATTRIBUTE PDU

Attribute PDUs have one of six types:

- Commands—PDUs sent to a server by a client that do not invoke a response.
- Requests—PDUs sent to a server by a client that invoke a response.
- Responses—PDUs sent to a client by a server in response to a request.
- Notifications—Unsolicited PDUs sent to a client by a server that do not invoke a confirmation.
- Indications—Unsolicited PDUs sent to a client by a server that invoke a confirmation.



- Confirmations—PDUs sent to a server by a client to confirm receipt of an indication.

A server shall be able to receive and properly respond to the following requests:

- *Find Information Request*
- *Read Request*

Support for all other PDU types in a server can be specified in a higher layer specification, see [Section 3.4.8](#).

If a client sends a request, then the client shall support all possible responses PDUs for that request.

If a server receives a request that it does not support, then the server shall respond with the *Error Response* with the Error Code «Request Not Supported», with the Attribute Handle In Error set to 0x0000.

If a server receives a command that it does not support, indicated by the Command Flag of the PDU set to one, then the server shall ignore the Command.

If the server receives an invalid request – for example, the PDU is the wrong length – then the server shall respond with the *Error Response* with the Error Code «Invalid PDU», with the Attribute Handle In Error set to 0x0000.

If a server does not have sufficient resources to process a request, then the server shall respond with the *Error Response* with the Error Code «Insufficient Resources», with the Attribute Handle In Error set to 0x0000.

If a server cannot process a request because an error was encountered during the processing of this request, then the server shall respond with the *Error Response* with the Error Code «Unlikely Error», with the Attribute Handle In Error set to 0x0000.

3.3.1 Attribute PDU Format

Attribute PDUs has the following format:

Name	Size (octets)	Description
Attribute Opcode	1	The attribute PDU operation code bit7: Authentication Signature Flag bit6: Command Flag bit5-0: Method

Table 3.1: Format of Attribute PDU



Name	Size (octets)	Description
Attribute Parameters	0 to (ATT_MTU - X)	The attribute PDU parameters X = 1 if Authentication Signature Flag of the Attribute Opcode is 0 X = 13 if Authentication Signature Flag of the Attribute Opcode is 1
Authentication Signature	0 or 12	Optional authentication signature for the Attribute Opcode and Attribute Parameters

Table 3.1: Format of Attribute PDU

Multi-octet fields within the attribute protocol shall be sent least significant octet first (little endian) with the exception of the Attribute Value field. The endianness of the Attribute Value field is defined by a higher layer specification.

The Attribute Opcode is composed of three fields, the Authentication Signature Flag, the Command Flag, and the Method. The Method is a 6-bit value that determines the format and meaning of the Attribute Parameters.

If the Authentication Signature Flag of the Attribute Opcode is set to one, the Authentication Signature value shall be appended to the end of the attribute PDU, and X is 13. If the Authentication Signature Flag of the Attribute Opcode is set to zero, the Authentication Signature value shall not be appended, and X is 1.

The Authentication Signature field is calculated as defined in Security Manager (see [\[Vol 3\] Part H, Section 2.4.5](#)). This value provides an Authentication Signature for the variable length message (m) consisting of the following values in this order: Attribute Opcode, Attribute Parameters.

An Attribute PDU that includes an Authentication Signature should not be sent on an encrypted link. Note: An encrypted link already includes authentication data on every packet and therefore adding more authentication data is not required.

If the Command Flag of the Attribute Opcode is set to one, the PDU shall be considered to be a Command.

Only the Write Command may include an Authentication Signature.

3.3.2 Sequential Protocol

Many attribute protocol PDUs use a sequential request-response protocol.

Once a client sends a request to a server, that client shall send no other request to the same server until a response PDU has been received.

Indications sent from a server also use a sequential indication-confirmation protocol. No other indications shall be sent to the same client from this server until a confirmation PDU has been received. The client, however, is free to send commands and requests prior to sending a confirmation.



For notifications, which do not have a response PDU, there is no flow control and a notification can be sent at any time.

Commands that do not require a response do not have any flow control. Note: A server can be flooded with commands, and a higher layer specification can define how to prevent this from occurring.

Commands and notifications that are received but cannot be processed, due to buffer overflows or other reasons, shall be discarded. Therefore, those PDUs must be considered to be unreliable.

Note: Flow control for each client and a server is independent.

Note: It is possible for a server to receive a request, send one or more notifications, and then the response to the original request. The flow control of requests is not affected by the transmission of the notifications.

Note: It is possible for a server to receive a request and then a command before responding to the original request. The flow control of requests is not affected by the transmission of commands.

Note: It is possible for a notification from a server to be sent after an indication has been sent but the confirmation has not been received. The flow control of indications is not affected by the transmission of notifications.

Note: It is possible for a client to receive an indication from a server and then send a request or command to that server before sending the confirmation of the original indication.

3.3.3 Transaction

An attribute protocol request and response or indication-confirmation pair is considered a single transaction. A transaction shall always be performed on one ATT Bearer, and shall not be split over multiple ATT Bearers.

On the client, a transaction shall start when the request is sent by the client. A transaction shall complete when the response is received by the client.

On a server, a transaction shall start when a request is received by the server. A transaction shall complete when the response is sent by the server.

On a server, a transaction shall start when an indication is sent by the server. A transaction shall complete when the confirmation is received by the server.

On a client, a transaction shall start when an indication is received by the client. A transaction shall complete when the confirmation is sent by the client.

A transaction not completed within 30 seconds shall time out. Such a transaction shall be considered to have failed and the local higher layers shall



be informed of this failure. No more attribute protocol requests, commands, indications or notifications shall be sent to the target device on this ATT Bearer.

Note: To send another attribute protocol PDU, a new ATT Bearer must be established between these devices. The existing ATT Bearer may need to be disconnected or the bearer terminated before the new ATT Bearer is established.

If the ATT Bearer is disconnected during a transaction, then the transaction shall be considered to be closed, and any values that were being modified on the server will be in an undetermined state, and any queue that was prepared by the client using this ATT Bearer shall be cleared.

Note: Each Prepare Write Request is a separate request and is therefore a separate transaction.

Note: Each Read Blob Request is a separate request and is therefore a separate transaction.

3.4 ATTRIBUTE PROTOCOL PDUS

3.4.1 Error Handling

3.4.1.1 Error Response

The *Error Response* is used to state that a given request cannot be performed, and to provide the reason.

Note: The Write Command does not generate an Error Response.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x01 = Error Response
Request Opcode In Error	1	The request that generated this error response
Attribute Handle In Error	2	The attribute handle that generated this error response
Error Code	1	The reason why the request has generated an error response

Table 3.2: Format of Error Response

The Request Opcode In Error parameter shall be set to the Attribute Opcode of the request that generated this error.

The Attribute Handle In Error parameter shall be set to the attribute handle in the original request that generated this error. If there was no attribute handle in



the original request or if the request is not supported, then the value 0x0000 shall be used for this field.

The Error Code parameter shall be set to one of the following values:

Name	Error Code	Description
Invalid Handle	0x01	The attribute handle given was not valid on this server.
Read Not Permitted	0x02	The attribute cannot be read.
Write Not Permitted	0x03	The attribute cannot be written.
Invalid PDU	0x04	The attribute PDU was invalid.
Insufficient Authentication	0x05	The attribute requires authentication before it can be read or written.
Request Not Supported	0x06	Attribute server does not support the request received from the client.
Invalid Offset	0x07	Offset specified was past the end of the attribute.
Insufficient Authorization	0x08	The attribute requires authorization before it can be read or written.
Prepare Queue Full	0x09	Too many prepare writes have been queued.
Attribute Not Found	0x0A	No attribute found within the given attribute handle range.
Attribute Not Long	0x0B	The attribute cannot be read using the Read Blob Request.
Insufficient Encryption Key Size	0x0C	The Encryption Key Size used for encrypting this link is insufficient.
Invalid Attribute Value Length	0x0D	The attribute value length is invalid for the operation.
Unlikely Error	0x0E	The attribute request that was requested has encountered an error that was unlikely, and therefore could not be completed as requested.
Insufficient Encryption	0x0F	The attribute requires encryption before it can be read or written.
Unsupported Group Type	0x10	The attribute type is not a supported grouping attribute as defined by a higher layer specification.
Insufficient Resources	0x11	Insufficient Resources to complete the request.
Reserved for future use	0x012 – 0x7F	Reserved for future use.

Table 3.3: Error Codes



Name	Error Code	Description
Application Error	0x80-0x9F	Application error code defined by a higher layer specification.
Reserved for future use	0xA0 – 0xDF	Reserved for future use
Common Profile and Service Error Codes	0xE0 – 0xFF	Common profile and service error codes defined in [Core Specification Supplement], Part B.

Table 3.3: Error Codes

If an error code is received in the *Error Response* that is not understood by the client, for example an error code that was reserved for future use that is now being used in a future version of this specification, then the *Error Response* shall still be considered to state that the given request cannot be performed for an unknown reason.

3.4.2 MTU Exchange

3.4.2.1 Exchange MTU Request

The *Exchange MTU Request* is used by the client to inform the server of the client's maximum receive MTU size and request the server to respond with its maximum receive MTU size.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x02 = Exchange MTU Request
Client Rx MTU	2	Client receive MTU size

Table 3.4: Format of Exchange MTU Request

The Client Rx MTU shall be greater than or equal to the default ATT_MTU.

This request shall only be sent once during a connection by the client. The Client Rx MTU parameter shall be set to the maximum size of the attribute protocol PDU that the client can receive.

3.4.2.2 Exchange MTU Response

The *Exchange MTU Response* is sent in reply to a received *Exchange MTU Request*.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x03 = Exchange MTU Response
Server Rx MTU	2	Attribute server receive MTU size

Table 3.5: Format of Exchange MTU Response



The Server Rx MTU shall be greater than or equal to the default ATT_MTU.

The Server Rx MTU parameter shall be set to the maximum size of the attribute protocol PDU that the server can receive.

The server and client shall set ATT_MTU to the minimum of the Client Rx MTU and the Server Rx MTU. The size is the same to ensure that a client can correctly detect the final packet of a long attribute read.

This ATT_MTU value shall be applied in the server after this response has been sent and before any other attribute protocol PDU is sent.

This ATT_MTU value shall be applied in the client after this response has been received and before any other attribute protocol PDU is sent.

If either Client Rx MTU or Service Rx MTU are incorrectly less than the default ATT_MTU, then the ATT_MTU shall not be changed and the ATT_MTU shall be the default ATT_MTU.

If a device is both a client and a server, the following rules shall apply:

1. A device's *Exchange MTU Request* shall contain the same MTU as the device's *Exchange MTU Response* (i.e. the MTU shall be symmetric).
2. If MTU is exchanged in one direction, that is sufficient for both directions.
3. It is permitted, (but not necessary - see 2.) to exchange MTU in both directions, but the MTUs shall be the same in each direction (see 1.)
4. If an Attribute Protocol Request is received after the MTU Exchange Request is sent and before the MTU Exchange Response is received, the associated Attribute Protocol Response shall use the default MTU. [Figure 3.1](#) shows an example that is covered by this rule. In this case device A and device B both use the default MTU for the Attribute Protocol Response.
5. Once the MTU Exchange Request has been sent, the initiating device shall not send an Attribute Protocol Indication or Notification until after the MTU Exchange Response has been received. Note: This stops the risk of a cross-over condition where the MTU size is unknown for the Indication or Notification.

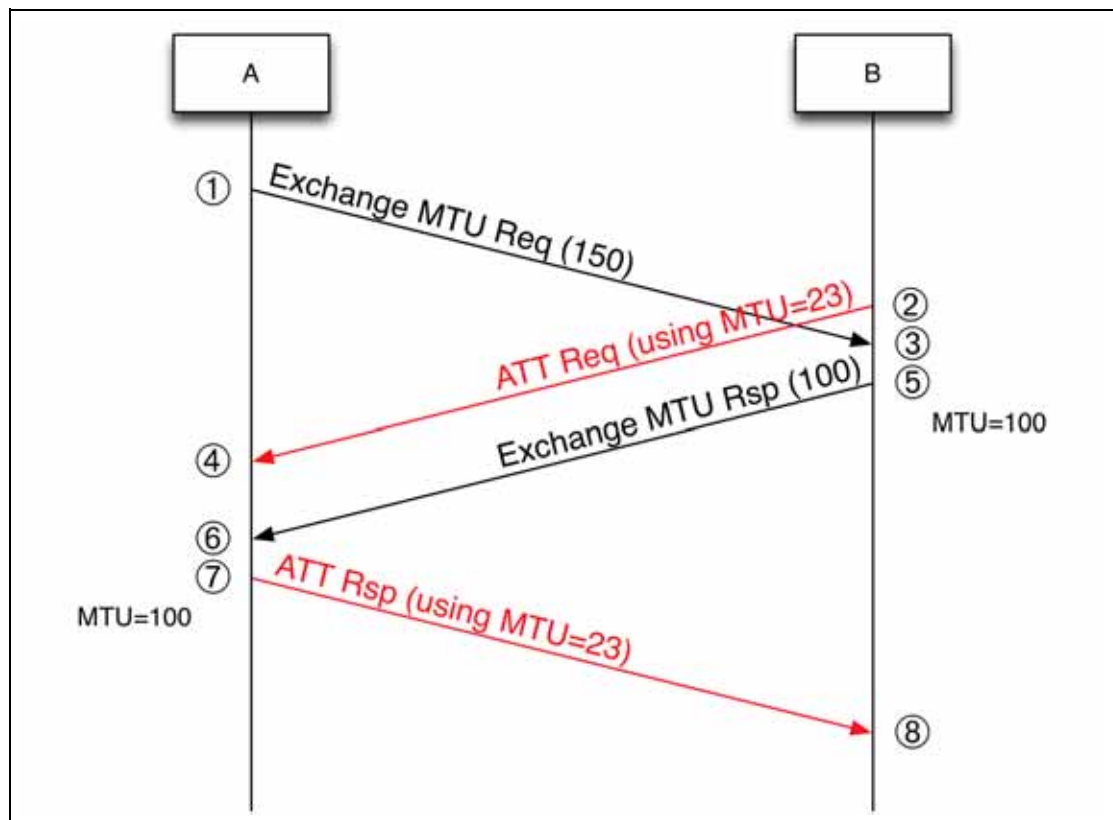


Figure 3.1: MTU request and response exchange

3.4.3 Find Information

3.4.3.1 Find Information Request

The *Find Information Request* is used to obtain the mapping of attribute handles with their associated types. This allows a client to discover the list of attributes and their types on a server.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x04 = Find Information Request
Starting Handle	2	First requested handle number
Ending Handle	2	Last requested handle number

Table 3.6: Format of Find Information Request

Only attributes with attribute handles between and including the Starting Handle parameter and the Ending Handle parameter will be returned. To read all attributes, the Starting Handle parameter shall be set to 0x0001, and the Ending Handle parameter shall be set to 0xFFFF. The Starting Handle parameter shall be less than or equal to the Ending Handle parameter.



If one or more attributes will be returned, a *Find Information Response* PDU shall be sent.

If a server receives a *Find Information Request* with the Starting Handle parameter greater than the Ending Handle parameter or the Starting Handle parameter is 0x0000, an *Error Response* shall be sent with the «Invalid Handle» error code; the Attribute Handle In Error parameter shall be set to the Starting Handle parameter.

If no attributes will be returned, an *Error Response* shall be sent with the «Attribute Not Found» error code; the Attribute Handle In Error parameter shall be set to the Starting Handle parameter.

The server shall not respond to the *Find Information Request* with an *Error Response* with the «Insufficient Authentication», «Insufficient Authorization», «Insufficient Encryption Key Size» or «Application Error» error code.

3.4.3.2 Find Information Response

The *Find Information Response* is sent in reply to a received *Find Information Request* and contains information about this server.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x05 = Find Information Response
Format	1	The format of the information data.
Information Data	4 to (ATT_MTU-2)	The information data whose format is determined by the Format field

Table 3.7: Format of Find Information Response

The *Find Information Response* shall have complete handle-UUID pairs. Such pairs shall not be split across response packets; this also implies that a handle-UUID pair shall fit into a single response packet. The handle-UUID pairs shall be returned in ascending order of attribute handles.

The Format parameter can contain one of two possible values.

Name	Format	Description
Handle(s) and 16-bit Bluetooth UUID(s)	0x01	A list of 1 or more handles with their 16-bit Bluetooth UUIDs
Handle(s) and 128-bit UUID(s)	0x02	A list of 1 or more handles with their 128-bit UUIDs

Table 3.8: Format field values

Attribute Protocol (ATT)

The information data field is comprised of a list of data defined in the tables below depending on the value chosen for the format.

Handle	16-bit Bluetooth UUID
2 octets	2 octets

Table 3.9: Format 0x01 - handle and 16-bit Bluetooth UUIDs

Handle	128-bit UUID
2 octets	16 octets

Table 3.10: Format 0x02 - handle and 128-bit UUIDs

Note: If sequential attributes have differing UUID sizes, it may happen that a *Find Information Response* is not filled with the maximum possible amount of (handle, UUID) pairs. This is because it is not possible to include attributes with differing UUID sizes into a single response packet. In that case, the following attribute would have to be read using another *Find Information Request* with its starting handle updated.



3.4.3.3 Find By Type Value Request

The *Find By Type Value Request* is used to obtain the handles of attributes that have a 16-bit UUID attribute type and attribute value. This allows the range of handles associated with a given attribute to be discovered when the attribute type determines the grouping of a set of attributes. Note: Generic Attribute Profile defines grouping of attributes by attribute type.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x06 = Find By Type Value Request
Starting Handle	2	First requested handle number
Ending Handle	2	Last requested handle number
Attribute Type	2	2 octet UUID to find
Attribute Value	0 to (ATT_MTU-7)	Attribute value to find

Table 3.11: Format of Find By Type Value Request

Only attributes with attribute handles between and including the Starting Handle parameter and the Ending Handle parameter that match the requested attribute type and the attribute value that have sufficient permissions to allow reading will be returned. To read all attributes, the Starting Handle parameter shall be set to 0x0001, and the Ending Handle parameter shall be set to 0xFFFF.

If one or more handles will be returned, a *Find By Type Value Response* PDU shall be sent.

Note: Attribute values will be compared in terms of length and binary representation.

Note: It is not possible to use this request on an attribute that has a value longer than (ATT_MTU-7).

If a server receives a *Find By Type Value Request* with the Starting Handle parameter greater than the Ending Handle parameter or the Starting Handle parameter is 0x0000, an *Error Response* shall be sent with the «Invalid Handle» error code. The Attribute Handle In Error parameter shall be set to the Starting Handle parameter.

If no attributes will be returned, an *Error Response* shall be sent by the server with the error code «Attribute Not Found». The Attribute Handle In Error parameter shall be set to the starting handle.

The server shall not respond to the *Find By Type Value Request* with an *Error Response* with the «Insufficient Authentication», «Insufficient Authorization», «Insufficient Encryption Key Size», «Insufficient Encryption» or «Application Error» error code.



3.4.3.4 Find By Type Value Response

The *Find By Type Value Response* is sent in reply to a received *Find By Type Value Request* and contains information about this server.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x07 = Find By Type Value Response
Handles Information List	4 to (ATT_MTU-1)	A list of 1 or more Handle Informations

Table 3.12: Format of Find By Type Value Response

The Handles Information List field is a list of one or more Handle Informations. The Handles Information field is an attribute handle range as defined in [Table 3.13](#).

Found Attribute Handle	Group End Handle
2 octets	2 octets

Table 3.13: Format of the Handles Information

The *Find By Type Value Response* shall contain one or more complete Handles Information. Such Handles Information shall not be split across response packets. The Handles Information List is ordered sequentially based on the found attribute handles.

For each handle that matches the attribute type and attribute value in the *Find By Type Value Request* a Handles Information shall be returned. The Found Attribute Handle shall be set to the handle of the attribute that has the exact attribute type and attribute value from the *Find By Type Value Request*. If the attribute type in the *Find By Type Value Request* is a grouping attribute as defined by a higher layer specification, the Group End Handle shall be defined by that higher layer specification. If the attribute type in the *Find By Type Value Request* is not a grouping attribute as defined by a higher layer specification, the Group End Handle shall be equal to the Found Attribute Handle.

Note: The Group End Handle may be greater than the Ending Handle in the *Find By Type Value Request*.

If a server receives a *Find By Type Value Request*, the server shall respond with the *Find By Type Value Response* containing as many handles for attributes that match the requested attribute type and attribute value that exist in the server that will fit into the maximum PDU size of (ATT_MTU-1).



3.4.4 Reading Attributes

3.4.4.1 Read By Type Request

The *Read By Type Request* is used to obtain the values of attributes where the attribute type is known but the handle is not known.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x08 = Read By Type Request
Starting Handle	2	First requested handle number
Ending Handle	2	Last requested handle number
Attribute Type	2 or 16	2 or 16 octet UUID

Table 3.14: Format of Read By Type Request

Only the attributes with attribute handles between and including the Starting Handle and the Ending Handle with the attribute type that is the same as the Attribute Type given will be returned. To search through all attributes, the starting handle shall be set to 0x0001 and the ending handle shall be set to 0xFFFF.

Note: All attribute types are effectively compared as 128-bit UUIDs, even if a 16-bit UUID is provided in this request or defined for an attribute. See [\[Vol 3\] Part B, Section 2.5.1](#).

The starting handle shall be less than or equal to the ending handle. If a server receives a *Read By Type Request* with the Starting Handle parameter greater than the Ending Handle parameter or the Starting Handle parameter is 0x0000, an *Error Response* shall be sent with the «Invalid Handle» error code; The Attribute Handle In Error parameter shall be set to the Starting Handle parameter.

If no attribute with the given type exists within the handle range, then no attribute handle and value will be returned, and an *Error Response* shall be sent with the error code «Attribute Not Found». The Attribute Handle In Error parameter shall be set to the starting handle.

The attributes returned shall be the attributes with the lowest handles within the handle range. These are known as the requested attributes.

If the attributes with the requested type within the handle range have attribute values that have the same length, then these attributes can all be read in a single request.

The attribute server shall include as many attributes as possible in the response in order to minimize the number of PDUs required to read attributes of the same type.



Note: If the attributes with the requested type within the handle range have attribute values with different lengths, then multiple *Read By Type Requests* must be made.

When multiple attributes match, then the rules below shall be applied to each in turn.

- Only attributes that can be read shall be returned in a *Read By Type Response*.
- If an attribute in the set of requested attributes would cause an *Error Response* then this attribute cannot be included in a *Read By Type Response* and the attributes before this attribute shall be returned.
- If the first attribute in the set of requested attributes would cause an *Error Response* then no other attributes in the requested attributes can be considered.

The server shall respond with a *Read By Type Response* if the requested attributes have sufficient permissions to allow reading.

If the client has insufficient authorization to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authorization». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

If the client has insufficient security to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authentication». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

If the client has an insufficient encryption key size to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Encryption Key Size». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

If the client has not enabled encryption, and encryption is required to read the requested attribute, then an *Error Response* shall be sent with the error code «Insufficient Encryption». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

If the requested attribute's value cannot be read due to permissions then an *Error Response* shall be sent with the error code «Read Not Permitted». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

Note: If there are multiple attributes with the requested type within the handle range, and the client would like to get the next attribute with the requested type, it would have to issue another *Read By Type Request* with its starting handle updated. The client can be sure there are no more such attributes remaining once it gets an *Error Response* with the error code «Attribute Not Found».



3.4.4.2 Read By Type Response

The *Read By Type Response* is sent in reply to a received *Read By Type Request* and contains the handles and values of the attributes that have been read.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x09 = Read By Type Response
Length	1	The size of each attribute handle-value pair
Attribute Data List	2 to (ATT_MTU - 2)	A list of Attribute Data

Table 3.15: Format of Read By Type Response

The *Read By Type Response* shall contain complete handle-value pairs. Such pairs shall not be split across response packets. The handle-value pairs shall be returned sequentially based on the attribute handle.

The Length parameter shall be set to the size of one attribute handle-value pair.

The maximum length of an attribute handle-value pair is 255 octets, bounded by the Length parameter that is one octet. Therefore, the maximum length of an attribute value returned in this response is $(\text{Length} - 2) = 253$ octets.

The attribute handle-value pairs shall be set to the value of the attributes identified by the attribute type within the handle range within the request. If the attribute value is longer than (ATT_MTU - 4) or 253 octets, whichever is smaller, then the first (ATT_MTU - 4) or 253 octets shall be included in this response.

Note: The *Read Blob Request* would be used to read the remaining octets of a long attribute value.

The Attribute Data field is comprised of a list of attribute handle and value pairs as defined in [Table 3.16](#).

Attribute Handle	Attribute Value
2 octets	$(\text{Length} - 2)$ octets

Table 3.16: Format of the Attribute Data



3.4.4.3 Read Request

The *Read Request* is used to request the server to read the value of an attribute and return its value in a *Read Response*.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x0A = Read Request
Attribute Handle	2	The handle of the attribute to be read

Table 3.17: Format of Read Request

The attribute handle parameter shall be set to a valid handle.

The server shall respond with a *Read Response* if the handle is valid and the attribute has sufficient permissions to allow reading.

If the client has insufficient authorization to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authorization».

If the client has insufficient security to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authentication».

If the client has an insufficient encryption key size to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Encryption Key Size».

If the client has not enabled encryption, and encryption is required to read the requested attribute, then an *Error Response* shall be sent with the error code «Insufficient Encryption».

If the handle is invalid, then an *Error Response* shall be sent with the error code «Invalid Handle».

If the attribute value cannot be read due to permissions then an *Error Response* shall be sent with the error code «Read Not Permitted».

3.4.4.4 Read Response

The read response is sent in reply to a received *Read Request* and contains the value of the attribute that has been read.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x0B = Read Response
Attribute Value	0 to (ATT_MTU-1)	The value of the attribute with the handle given

Table 3.18: Format of Read Response



The attribute value shall be set to the value of the attribute identified by the attribute handle in the request. If the attribute value is longer than (ATT_MTU-1) then the first (ATT_MTU-1) octets shall be included in this response.

Note: The *Read Blob Request* would be used to read the remaining octets of a long attribute value.

3.4.4.5 Read Blob Request

The *Read Blob Request* is used to request the server to read part of the value of an attribute at a given offset and return a specific part of the value in a *Read Blob Response*.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x0C = Read Blob Request
Attribute Handle	2	The handle of the attribute to be read
Value Offset	2	The offset of the first octet to be read

Table 3.19: Format of Read Blob Request

The attribute handle parameter shall be set to a valid handle.

The value offset parameter is based from zero; the first value octet has an offset of zero, the second octet has a value offset of one, etc.

The server shall respond with a *Read Blob Response* if the handle is valid and the attribute and value offset is not greater than the length of the attribute value and has sufficient permissions to allow reading.

If the client has insufficient authorization to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authorization».

If the client has insufficient security to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authentication».

If the client has an insufficient encryption key size to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Encryption Key Size».

If the client has not enabled encryption, and encryption is required to read the requested attribute, then an *Error Response* shall be sent with the error code «Insufficient Encryption».

If the handle is invalid, then an *Error Response* shall be sent with the error code «Invalid Handle».

If the attribute value cannot be read due to permissions then an *Error Response* shall be sent with the error code «Read Not Permitted».



If the value offset of the *Read Blob Request* is greater than the length of the attribute value, an *Error Response* shall be sent with the error code «Invalid Offset».

If the attribute value has a fixed length that is less than or equal to (ATT_MTU - 1) octets in length, then an *Error Response* may be sent with the error code «Attribute Not Long».

If the value offset of the *Read Blob Request* is equal to the length of the attribute value, then the length of the part attribute value in the response shall be zero.

Note: If the attribute is longer than (ATT_MTU-1) octets, the Read Blob Request is the only way to read the additional octets of a long attribute. The first (ATT_MTU-1) octets may be read using a *Read Request*, an *Handle Value Notification* or an *Handle Value Indication*.

Note: Long attributes may or may not have their length specified by a higher layer specification. If the long attribute has a variable length, the only way to get to the end of it is to read it part by part until the value in the *Read Blob Response* has a length shorter than (ATT_MTU-1) or an *Error Response* with the error code «Invalid Offset».

Note: The value of a Long Attribute may change between one Read Blob Request and the next Read Blob Request. A higher layer specification should be aware of this and define appropriate behavior.

3.4.4.6 Read Blob Response

The *Read Blob Response* is sent in reply to a received *Read Blob Request* and contains part of the value of the attribute that has been read.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x0D = Read Blob Response
Part Attribute Value	0 to (ATT_MTU - 1)	Part of the value of the attribute with the handle given

Table 3.20: Format of Read Blob Response

The part attribute value shall be set to part of the value of the attribute identified by the attribute handle and the value offset in the request. If the value offset is equal to the length of the attribute value, then the length of the part attribute value shall be zero. If the attribute value is longer than (Value Offset + ATT_MTU-1) then (ATT_MTU-1) octets from Value Offset shall be included in this response.



3.4.4.7 Read Multiple Request

The *Read Multiple Request* is used to request the server to read two or more values of a set of attributes and return their values in a *Read Multiple Response*. Only values that have a known fixed size can be read, with the exception of the last value that can have a variable length. The knowledge of whether attributes have a known fixed size is defined in a higher layer specification.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x0E = Read Multiple Request
Set Of Handles	4 to (ATT_MTU - 1)	A set of two or more attribute handles.

Table 3.21: Format of Read Multiple Request

The attribute handles in the Set Of Handles parameter shall be valid handles.

The server shall respond with a *Read Multiple Response* if all the handles are valid and all attributes have sufficient permissions to allow reading.

Note: The attribute values for the attributes in the Set Of Handles parameters do not have to all be the same size.

Note: The attribute handles in the Set Of Handles parameter do not have to be in attribute handle order; they are in the order that the values are required in the response.

If the client has insufficient authorization to read any of the attributes then an *Error Response* shall be sent with the error code «Insufficient Authorization».

If the client has insufficient security to read any of the attributes then an *Error Response* shall be sent with the error code «Insufficient Authentication».

If the client has an insufficient encryption key size to read any of the attributes then an *Error Response* shall be sent with the error code «Insufficient Encryption Key Size».

If the client has not enabled encryption, and encryption is required to read the requested attribute, then an *Error Response* shall be sent with the error code «Insufficient Encryption».

If any of the handles are invalid, then an *Error Response* shall be sent with the error code «Invalid Handle».

If any of the attribute values cannot be read due to permissions then an *Error Response* shall be sent with the error code «Read Not Permitted».

If an *Error Response* is sent, the Attribute Handle In Error parameter shall be set to the handle of the first attribute causing the error.



3.4.4.8 Read Multiple Response

The read response is sent in reply to a received *Read Multiple Request* and contains the values of the attributes that have been read.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x0F = Read Multiple Response
Set Of Values	0 to (ATT_MTU-1)	A set of two or more values

Table 3.22: Format of Read Multiple Response

The Set Of Values parameter shall be a concatenation of attribute values for each of the attribute handles in the request in the order that they were requested. If the Set Of Values parameter is longer than (ATT_MTU-1) then only the first (ATT_MTU-1) octets shall be included in this response.

Note: A client should not use this request for attributes when the Set Of Values parameter could be (ATT_MTU-1) as it will not be possible to determine if the last attribute value is complete, or if it overflowed.

3.4.4.9 Read by Group Type Request

The *Read By Group Type Request* is used to obtain the values of attributes where the attribute type is known, the type of a grouping attribute as defined by a higher layer specification, but the handle is not known.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x10 = Read By Group Type Request
Starting Handle	2	First requested handle number
Ending Handle	2	Last requested handle number
Attribute Group Type	2 or 16	2 or 16 octet UUID

Table 3.23: Format of Read By Group Type Request

Only the attributes with attribute handles between and including the Starting Handle and the Ending Handle with the attribute type that is the same as the Attribute Group Type given will be returned. To search through all attributes, the starting handle shall be set to 0x0001 and the ending handle shall be set to 0xFFFF.

Note: All attribute types are effectively compared as 128-bit UUIDs, even if a 16-bit UUID is provided in this request or defined for an attribute. See [\[Vol 3\] Part B, Section 2.5.1](#).

The starting handle shall be less than or equal to the ending handle. If a server receives a *Read By Group Type Request* with the Starting Handle parameter greater than the Ending Handle parameter or the Starting Handle parameter is



0x0000, an *Error Response* shall be sent with the «Invalid Handle» error code; The Attribute Handle In Error parameter shall be set to the Starting Handle parameter.

If the Attribute Group Type is not a supported grouping attribute as defined by a higher layer specification then an *Error Response* shall be sent with the error code «Unsupported Group Type». The Attribute Handle In Error parameter shall be set to the Starting Handle.

If no attribute with the given type exists within the handle range, then no attribute handle and value will be returned, and an *Error Response* shall be sent with the error code «Attribute Not Found». The Attribute Handle In Error parameter shall be set to the starting handle.

The attributes returned shall be the attributes with the lowest handles within the handle range. These are known as the requested attributes.

If the attributes with the requested type within the handle range have attribute values that have the same length, then these attributes can all be read in a single request.

The attribute server shall include as many attributes as possible in the response in order to minimize the number of PDUs required to read attributes of the same type.

Note: If the attributes with the requested type within the handle range have attribute values with different lengths, then multiple *Read By Group Type Requests* must be made.

When multiple attributes match, then the rules below shall be applied to each in turn.

- Only attributes that can be read shall be returned in a *Read By Group Type Response*.
- If an attribute in the set of requested attributes would cause an *Error Response* then this attribute cannot be included in a *Read By Group Type Response* and the attributes before this attribute shall be returned.
- If the first attribute in the set of requested attributes would cause an *Error Response* then no other attributes in the requested attributes can be considered.

The server shall respond with a *Read By Group Type Response* if the requested attributes have sufficient permissions to allow reading.

If the client has insufficient authorization to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authorization». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.



If the client has insufficient security to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authentication». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

If the client has an insufficient encryption key size to read the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Encryption Key Size». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

If the client has not enabled encryption, and encryption is required to read the requested attribute, then an *Error Response* shall be sent with the error code «Insufficient Encryption». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

If the requested attribute's value cannot be read due to permissions then an *Error Response* shall be sent with the error code «Read Not Permitted». The Attribute Handle In Error parameter shall be set to the handle of the attribute causing the error.

Note: If there are multiple attributes with the requested type within the handle range, and the client would like to get the next attribute with the requested type, it would have to issue another *Read By Group Type Request* with its starting handle updated. The client can be sure there are no more such attributes remaining once it gets an *Error Response* with the error code «Attribute Not Found».

3.4.4.10 Read by Group Type Response

The *Read By Group Type Response* is sent in reply to a received *Read By Group Type Request* and contains the handles and values of the attributes that have been read.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x11 = Read By Group Type Response
Length	1	The size of each Attribute Data
Attribute Data List	4 to (ATT_MTU - 2)	A list of Attribute Data

Table 3.24: Format of Read By Group Type Response

The *Read By Group Type Response* shall contain complete Attribute Data. An Attribute Data shall not be split across response packets. The Attribute Data List is ordered sequentially based on the attribute handles

The Length parameter shall be set to the size of the one Attribute Data.

The maximum length of an Attribute Data is 255 octets, bounded by the Length parameter that is one octet. Therefore, the maximum length of an attribute value returned in this response is (Length – 4) = 251 octets.



The Attribute Data List shall be set to the value of the attributes identified by the attribute type within the handle range within the request. If the attribute value is longer than (ATT_MTU - 6) or 251 octets, whichever is smaller, then the first (ATT_MTU - 6) or 251 octets shall be included in this response.

Note: The *Read Blob Request* would be used to read the remaining octets of a long attribute value.

The Attribute Data List is comprised of a list of Attribute Data as defined in [Table 3.25](#).

Attribute Handle	End Group Handle	Attribute Value
2 octets	2 octets	(Length - 4) octets

Table 3.25: Format of the Attribute Data

3.4.5 Writing Attributes

3.4.5.1 Write Request

The *Write Request* is used to request the server to write the value of an attribute and acknowledge that this has been achieved in a *Write Response*.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x12 = Write Request
Attribute Handle	2	The handle of the attribute to be written
Attribute Value	0 to (ATT_MTU-3)	The value to be written to the attribute

Table 3.26: Format of Write Request

The Attribute Handle shall be set to a valid handle.

The Attribute Value shall be set to the new value of the attribute.

If the attribute value has a variable length, then the attribute value shall be truncated or lengthened to match the length of the Attribute Value parameter.

Note: If an attribute value has a variable length and if the Attribute Value parameter is of zero length, the attribute value will be fully truncated.

If the attribute value has a fixed length and the Attribute Value parameter length is less than or equal to the length of the attribute value, the octets of the attribute value parameter length shall be written; all other octets in this attribute value shall be unchanged.



The server shall respond with a *Write Response* if the handle is valid, the attribute has sufficient permissions to allow writing, and the attribute value has a valid size and format, and it is successful in writing the attribute.

If the attribute value has a variable length and the Attribute Value parameter length exceeds the maximum valid length of the attribute value then the server shall respond with an *Error Response* with the error code «Invalid Attribute Value Length».

If the attribute value has a fixed length and the requested attribute value parameter length is greater than the length of the attribute value then the server shall respond with an *Error Response* with the error code «Invalid Attribute Value Length».

If the client has insufficient authorization to write the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authorization».

If the client has insufficient security to write the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authentication».

If the client has an insufficient encryption key size to write the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Encryption Key Size».

If the client has not enabled encryption, and encryption is required to write the requested attribute, then an *Error Response* shall be sent with the error code «Insufficient Encryption».

If the handle is invalid, then an *Error Response* shall be sent with the error code «Invalid Handle».

If the attribute value cannot be written due to permissions then an *Error Response* shall be sent with the error code «Write Not Permitted».

If the attribute value cannot be written due to an application error then an *Error Response* shall be sent with an error code defined by a higher layer specification.

3.4.5.2 Write Response

The *Write Response* is sent in reply to a valid *Write Request* and acknowledges that the attribute has been successfully written.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x13 = Write Response

Table 3.27: Format of Write Response

The *Write Response* shall be sent after the attribute value is written.



3.4.5.3 Write Command

The *Write Command* is used to request the server to write the value of an attribute, typically into a control-point attribute.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x52 = Write Command
Attribute Handle	2	The handle of the attribute to be set
Attribute Value	0 to (ATT_MTU-3)	The value of be written to the attribute

Table 3.28: Format of Write Command

The attribute handle parameter shall be set to a valid handle.

The attribute value parameter shall be set to the new value of the attribute.

If the attribute value has a variable length, then the attribute value shall be truncated or lengthened to match the length of the attribute value parameter.

Note: If an attribute value has a variable length and if the attribute value parameter is of zero length, the attribute value will be fully truncated.

If the attribute value has a fixed length and the attribute value parameter length is less than or equal to the length of the attribute value, the octets up to the attribute value parameter length shall be written; all other octets in this attribute value shall be unchanged.

If the attribute value has a variable length and the attribute value parameter length exceeds the maximum valid length of the attribute value then the server shall ignore the command.

If the attribute value has a fixed length and the requested attribute value parameter length is greater than the length of the attribute value then the server shall ignore the command.

No *Error Response* or *Write Response* shall be sent in response to this command. If the server cannot write this attribute for any reason the command shall be ignored.



3.4.5.4 Signed Write Command

The *Signed Write Command* is used to request the server to write the value of an attribute with an authentication signature, typically into a control-point attribute.

Parameter	Size (Octets)	Description
Attribute Opcode	1	0xD2 = Signed Write Command
Attribute Handle	2	The handle of the attribute to be set
Attribute Value	0 to (ATT_MTU - 15)	The value to be written to the attribute
Authentication Signature	12	Authentication signature for the Attribute Opcode, Attribute Handle and Attribute Value Parameters

Table 3.29: Format of Signed Write Command

The attribute handle parameter shall be set to a valid handle.

The attribute value parameter shall be set to the new value of the attribute.

The attribute signature shall be calculated as defined in [Section 3.3.1](#).

For example, if the variable length message *m* to be signed is 'D212001337', SignCounter is 0x00000001 and key is 0x611B64EBFBCD1FD372EC9196DF425E50, then message to be signed (*M*) by the CMAC function is the octet sequence 'D21200133701000000'. The padding(*M*) is 0x0000000137130012D280000000000000, resultant CMAC is 0xF20F903C931E87F159B64F012574B4D0 and Authentication Signature is the octet sequence '01000000F1871E933C900FF2'. The final signed message is 'D21200133701000000F1871E933C900FF2'.

If the attribute value has a variable length, then the attribute value shall be truncated or lengthened to match the length of the attribute value parameter.

Note: If an attribute value has a variable length and if the attribute value parameter is of zero length, the attribute value will be fully truncated.

If the attribute value has a fixed length and the attribute value parameter length is less than or equal to the length of the attribute value, the octets up to the attribute value parameter length shall be written; all other octets in this attribute value shall be unchanged.

If the attribute value has a variable length and the attribute value parameter length exceeds the maximum valid length of the attribute value then the server shall ignore the command.

If the attribute value has a fixed length and the requested attribute value parameter length is greater than the length of the attribute value then the server shall ignore the command.



If the authentication signature verification fails, then the server shall ignore the command.

No *Error Response* or *Write Response* shall be sent in response to this command. If the server cannot write this attribute for any reason the command shall be ignored.

3.4.6 Queued Writes

The purpose of queued writes is to queue up writes of values of multiple attributes in a first-in first-out queue and then execute the write on all of them in a single atomic operation.

3.4.6.1 Prepare Write Request

The *Prepare Write Request* is used to request the server to prepare to write the value of an attribute. The server will respond to this request with a *Prepare Write Response*, so that the client can verify that the value was received correctly.

A client may send more than one *Prepare Write Request* to a server, which will queue and send a response for each handle value pair.

A server may limit the number of prepare write requests that it can accept. A higher layer specification should define this limit.

After a *Prepare Write Request* has been issued, and the response received, any other attribute command or request can be issued from the same client to the same server.

Each client's queued values are separate; the execution of one queue shall not affect the preparation or execution of any other client's queued values.

Any actions on attributes that exist in the prepare queue shall proceed as if the prepare queue did not exist, and the prepare queue shall be unaffected by these actions. A subsequent execute write will write the values in the prepare queue even if the value of the attribute has changed since the prepare writes were started.

The attribute protocol makes no determination on the validity of the Part Attribute Value or the Value Offset. A higher layer specification determines the meaning of the data.

Each *Prepare Write Request* will be queued even if the attribute handle is the same as a previous *Prepare Write Request*. These will then be executed in the order received, causing multiple writes for this attribute to occur.



If the link is lost while a number of prepared write requests have been queued, the queue will be cleared and no writes will be executed.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x16 = Prepare Write Request
Attribute Handle	2	The handle of the attribute to be written
Value Offset	2	The offset of the first octet to be written
Part Attribute Value	0 to (ATT_MTU-5)	The value of the attribute to be written

Table 3.30: Format of Prepare Write Request

The Attribute Handle parameter shall be set to a valid handle.

The Value Offset parameter shall be set to the offset of the first octet where the Part Attribute Value parameter is to be written within the attribute value. The Value Offset parameter is based from zero; the first octet has an offset of zero, the second octet has an offset of one, etc.

The server shall respond with a *Prepare Write Response* if the handle is valid, the attribute has sufficient permissions to allow writing at this time, and the prepare queue has sufficient space.

Note: The Attribute Value validation is done when an Execute Write Request is received. Hence, any Invalid Offset or Invalid Attribute Value Length errors are generated when an Execute Write Request is received.

If the client has insufficient authorization to write the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authorization».

If the client has insufficient security to write the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Authentication».

If the client has an insufficient encryption key size to write the requested attribute then an *Error Response* shall be sent with the error code «Insufficient Encryption Key Size».

If the client has not enabled encryption, and encryption is required to write the requested attribute, then an *Error Response* shall be sent with the error code «Insufficient Encryption».

If the server does not have sufficient space to queue this request then an *Error Response* shall be sent with the error code «Prepare Queue Full».

If the handle is invalid, then an *Error Response* shall be sent with the error code «Invalid Handle».



If the attribute value cannot be written then an *Error Response* shall be sent with the error code «Write Not Permitted».

The server shall not change the value of the attribute until an *Execute Write Request* is received.

If a *Prepare Write Request* was invalid, and therefore an *Error Response* has been issued, then this prepared write will be considered to have not been received. All existing prepared writes in the prepare queue shall not be affected by this invalid request.

3.4.6.2 Prepare Write Response

The *Prepare Write Response* is sent in response to a received *Prepare Write Request* and acknowledges that the value has been successfully received and placed in the prepare write queue.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x17 = Prepare Write Response
Attribute Handle	2	The handle of the attribute to be written
Value Offset	2	The offset of the first octet to be written
Part Attribute Value	0 to (ATT_MTU-5)	The value of the attribute to be written

Table 3.31: Format of Prepare Write Response

The attribute handle shall be set to the same value as in the corresponding *Prepare Write Request*.

The value offset and part attribute value shall be set to the same values as in the corresponding *Prepare Write Request*.

3.4.6.3 Execute Write Request

The *Execute Write Request* is used to request the server to write or cancel the write of all the prepared values currently held in the prepare queue from this client. This request shall be handled by the server as an atomic operation.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x18 = Execute Write Request

Table 3.32: Format of Execute Write Request



Parameter	Size (octets)	Description
Flags	1	0x00 – Cancel all prepared writes 0x01 – Immediately write all pending prepared values

Table 3.32: Format of Execute Write Request

When the flags parameter is set to 0x01, values that were queued by the previous prepare write requests shall be written in the order they were received in the corresponding *Prepare Write Request*. The queue shall then be cleared, and an *Execute Write Response* shall be sent.

When the flags parameter is set to 0x00 all pending prepare write values shall be discarded for this client. The queue shall then be cleared, and an *Execute Write Response* shall be sent.

If there are no pending prepared write values, then no values are written, and an *Execute Write Response* shall be sent.

If the prepared Attribute Value exceeds the maximum valid length of the attribute value then all pending prepare write values shall be discarded for this client, the queue shall then be cleared, and an *Error Response* shall be sent with the error code «Invalid Attribute Value Length».

If the prepare Value Offset is greater than the current length of the attribute value then all pending prepare write values shall be discarded for this client, the queue shall be cleared and then an *Error Response* shall be sent with the «Invalid Offset».

If the prepare write requests cannot be written, due to an application error, the queue shall be cleared and then an *Error Response* shall be sent with a higher layer specification defined error code. The Attribute Handle In Error parameter shall be set to the attribute handle of the attribute from the prepare queue that caused this application error. The state of the attributes that were to be written from the prepare queue is not defined in this case.

3.4.6.4 Execute Write Response

The *Execute Write Response* is sent in response to a received *Execute Write Request*.

Parameter	Size	Description
Attribute Opcode	1	0x19 - Execute Write Response

Table 3.33: Format of Execute Write Response

The *Execute Write Response* shall be sent after the attributes are written. In case an action is taken in response to the write, an indication may be used once the action is complete.



3.4.7 Server Initiated

3.4.7.1 Handle Value Notification

A server can send a notification of an attribute's value at any time.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x1B = Handle Value Notification
Attribute Handle	2	The handle of the attribute
Attribute Value	0 to (ATT_MTU-3)	The current value of the attribute

Table 3.34: Format of Handle Value Notification

The attribute handle shall be set to a valid handle.

The attribute value shall be set to the current value of attribute identified by the attribute handle.

If the attribute value is longer than (ATT_MTU-3) octets, then only the first (ATT_MTU-3) octets of this attributes value can be sent in a notification.

Note: For a client to get a long attribute, it would have to use the *Read Blob Request* following the receipt of this notification.

If the attribute handle or the attribute value is invalid, then this notification shall be ignored upon reception.

3.4.7.2 Handle Value Indication

A server can send an indication of an attribute's value.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x1D = Handle Value Indication
Attribute Handle	2	The handle of the attribute
Attribute Value	0 to (ATT_MTU-3)	The current value of the attribute

Table 3.35: Format of Handle Value Indication

The attribute handle shall be set to a valid handle.

The attribute value shall be set to the current value of attribute identified by the attribute handle.

If the attribute value is longer than (ATT_MTU-3) octets, then only the first (ATT_MTU - 3) octets of this attributes value can be sent in an indication.



Note: For a client to get a long attribute, it would have to use the *Read Blob Request* following the receipt of this indication.

The client shall send a *Handle Value Confirmation* in response to a *Handle Value Indication*. No further indications to this client shall occur until the confirmation has been received by the server.

If the attribute handle or the attribute value is invalid, the client shall send a handle value confirmation in response and shall discard the handle and value from the received indication.

3.4.7.3 Handle Value Confirmation

The *Handle Value Confirmation* is sent in response to a received *Handle Value Indication* and confirms that the client has received an indication of the given attribute.

Parameter	Size (octets)	Description
Attribute Opcode	1	0x1E = Handle Value Confirmation

Table 3.36: Format of Handle Value Confirmation



3.4.8 Attribute Opcode Summary

Table 3.37 gives a summary of the attribute protocol PDUs. .

Attribute PDU Name	Attribute Opcode	Parameters
Error Response	0x01	Request Opcode in Error, Attribute Handle In Error, Error Code
Exchange MTU Request	0x02	Client Rx MTU
Exchange MTU Response	0x03	Server Rx MTU
Find Information Request	0x04	Starting Handle, Ending Handle
Find Information Response	0x05	Format, Information Data
Find By Type Value Request	0x06	Starting Handle, Ending Handle, Attribute Type, Attribute Value
Find By Type Value Response	0x07	Handles Information List
Read By Type Request	0x08	Starting Handle, Ending Handle, UUID
Read By Type Response	0x09	Length, Attribute Data List
Read Request	0x0A	Attribute Handle
Read Response	0x0B	Attribute Value
Read Blob Request	0x0C	Attribute Handle, Value Offset
Read Blob Response	0x0D	Part Attribute Value
Read Multiple Request	0x0E	Handle Set
Read Multiple Response	0x0F	Value Set
Read by Group Type Request	0x10	Start Handle, Ending Handle, UUID
Read by Group Type Response	0x11	Length, Attribute Data List
Write Request	0x12	Attribute Handle, Attribute Value
Write Response	0x13	-

Table 3.37: Attribute Protocol Summary

Attribute Protocol (ATT)

Attribute PDU Name	Attribute Opcode	Parameters
Write Command	0x52	Attribute Handle, Attribute Value
Prepare Write Request	0x16	Attribute Handle, Value Offset, Part Attribute Value
Prepare Write Response	0x17	Attribute Handle, Value Offset, Part Attribute Value
Execute Write Request	0x18	Flags
Execute Write Response	0x19	-
Handle Value Notification	0x1B	Attribute Handle, Attribute Value
Handle Value Indication	0x1D	Attribute Handle, Attribute Value
Handle Value Confirmation	0x1E	
Signed Write Command	0xD2	Attribute Handle, Attribute Value, Authentication Signature

Table 3.37: Attribute Protocol Summary



3.4.9 Attribute PDU Response Summary

Table 3.38 gives a summary of the Attribute PDU Method responses that are allowed. Each method indicates the method that should be sent as a successful response, and whether an Error Response can be sent in response instead. If an Error Response can be sent, then the table also indicates the Error Codes that are valid within this Error Response for the given method.

Attribute PDU Method	Successful Response	Error Response Allowed	Error Response Error Codes
Exchange MTU Request	Exchange MTU Response	Yes	Request Not Supported
Find Information Request	Find Information Response	Yes	Invalid Handle, Attribute Not Found
Find By Type Value Request	Find By Type Value Response	Yes	Invalid Handle, Request Not Supported, Attribute Not Found
Read By Type Request	Read By Type Response	Yes	Invalid Handle, Request Not Supported, Attribute Not Found, Insufficient Authorization, Insufficient Authentication, Insufficient Encryption, Insufficient Encryption Key Size, Read Not Permitted, Application Error
Read Request	Read Response	Yes	Invalid Handle, Insufficient Authorization, Insufficient Authentication, Insufficient Encryption, Insufficient Encryption Key Size, Read Not Permitted, Application Error

Table 3.38: Attribute Request and Response Summary



Attribute PDU Method	Successful Response	Error Response Allowed	Error Response Error Codes
Read Blob Request	Read Blob Response	Yes	Invalid Handle, Request Not Supported, Insufficient Authorization, Insufficient Authentication, Insufficient Encryption, Insufficient Encryption Key Size, Read Not Permitted, Invalid Offset, Attribute Not Long, Application Error
Read Multiple Request	Read Multiple Response	Yes	Invalid Handle, Request Not Supported, Insufficient Authorization, Insufficient Authentication, Insufficient Encryption, Insufficient Encryption Key Size, Read Not Permitted, Application Error
Read by Group Type Request	Read by Group Type Response	Yes	Invalid Handle, Request Not Supported, Attribute Not Found, Insufficient Authorization, Insufficient Authentication, Insufficient Encryption, Insufficient Encryption Key Size Read Not Permitted, Unsupported Group Type, Application Error
Write Request	Write Response	Yes	Invalid Handle, Request Not Supported, Insufficient Authorization, Insufficient Authentication, Insufficient Encryption, Insufficient Encryption Key Size, Write Not Permitted, Invalid Attribute Value Length, Application Error

Table 3.38: Attribute Request and Response Summary

Attribute Protocol (ATT)



Attribute PDU Method	Successful Response	Error Response Allowed	Error Response Error Codes
Write Command	N/A	No	
Signed Write Command	N/A	No	
Prepare Write Request	Prepare Write Response	Yes	Invalid Handle, Request Not Supported, Insufficient Authorization, Insufficient Authentication, Write Not Permitted, Prepare Queue Full, Insufficient Encryption, Insufficient Encryption Key Size, Application Error
Execute Write Request	Execute Write Response	Yes	Application Error, Invalid Offset, Invalid Attribute Value Length
Handle Value Notification	N/A	No	
Handle Value Indication	Handle Value Confirmation	No	

Table 3.38: Attribute Request and Response Summary



4 SECURITY CONSIDERATIONS

The attribute protocol can be used to access information that may require both authorization and an authenticated and encrypted physical link before an attribute can be read or written.

If such a request is issued when the client has not been authorized to access this information, the server shall send an *Error Response* with the error code set to «Insufficient Authorization». The authorization requirements for access to a given attribute are not defined in this specification. Each device implementation will determine how authorization occurs. Authorization procedures are defined in GAP, and may be further refined in a higher layer specification.

If such a request is issued when the physical link is unauthenticated, the server shall send an *Error Response* with the error code set to «Insufficient Authentication». A client wanting to read or write this attribute can then request that the physical link be authenticated, and once this has been completed, send the request again.

The attribute protocol can be used to notify or indicate the value of an attribute that may require an authenticated and encrypted physical link before an attribute notification or indication is performed. A server wanting to notify or indicate this attribute can then request that the physical link be authenticated, and once this has been completed, send the notification or indication.

The list of attributes that a device supports is not considered private or confidential information, and therefore the *Find Information Request* shall always be permitted. This implies that an «Insufficient Authorization» or «Insufficient Authentication» error code shall not be used in an *Error Response* for a *Find Information Request*.

For example, an attribute value may be allowed to be read by any device, but only written by an authenticated device. An implementation should take this into account, and not assume that just because it can read an attribute's value, it will also be able to write the value. Similarly, just because an attribute value can be written, does not mean that an attribute value can also be read. Each individual attribute could have different security requirements.

When a client accesses an attribute, the order of checks that are performed on the server will have security implications. A server shall check authentication and authorization requirements before any other check is performed.

Note: For example, if the authentication and authorization requirement checks are not performed first then the size of an attribute could be determined by performing repeated read blob requests on an attribute that a client does not have access to, because either an «Invalid Offset» error code or «Insufficient Authentication» error codes would be returned.